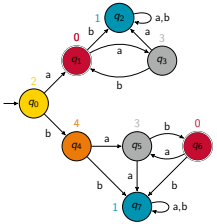
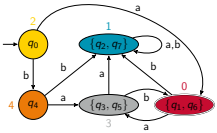
Module 7 project:
Similarity and Symmetry for Discrete Structures

Part III: Fast Equivalence Testing and Minimization of DFAs





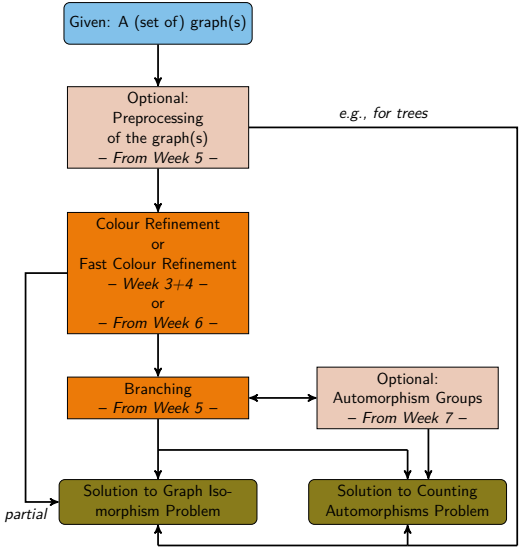
Ruben Hoeksma

Outline

This week:

- A colour refinement algorithm (Fast Colour Refinement) that (with correct implementation and smart use of the correct data structures) is faster than the basic colour refinement algorithm.
- Explanation through *similarity testing*: for *testing equivalence for DFAs*, and *minimizing DFAs*.
- Discussion on how to translate to GI/#Aut problems at the end of this lecture.
- A very efficient implementation of this algorithm: $O(m \log n)$, where:
  - n is the number of states, and
  - m is the total number of transitions.
  - Some data structure considerations.

Overview (also Page 4 of the reader)



DFAs: Equivalent States

Consider a DFA (Deterministic Finite Automaton) $D = (Q, \Sigma, \delta, q_0, F)$.
Minimization:
Finding a possibly smaller DFA that accepts the same language.

- First step: unreachable states can easily be removed.
- Second step: 'Equivalent states' can be identified into one state.

Definition

Two states q_i and q_j are equivalent if they accept the same language. That is:
$\mathcal{L}((Q, \Sigma, \delta, q_i, F)) = \mathcal{L}(Q, \Sigma, \delta, q_j, F)$.

In other words: q_i and q_j are *not equivalent* iff there exists a word w that *distinguishes* them:

$$\hat{\delta}(q_i, w) \in F \leftrightarrow \hat{\delta}(q_j, w) \notin F.$$

(In words: either $\hat{\delta}(q_i, w) \in F$ and $\hat{\delta}(q_j, w) \notin F$, or $\hat{\delta}(q_i, w) \notin F$ and $\hat{\delta}(q_j, w) \in F$.)

DFAs: minimization and testing equivalence

Goal: Find the partition into equivalent states.

Method: Partition/colour refinement: start with a valid partition, and refine this until it is *stable*.

- DFA minimization: identify equivalent states into one state.
- DFA equivalence testing: apply the method to two DFAs D_1 and D_2 simultaneously.
  Then $\mathcal{L}(D_1) = \mathcal{L}(D_2)$ if and only if their two start states are equivalent.

Throughout, we will work with a partition / colouring of the states Q of a DFA.
Notations:

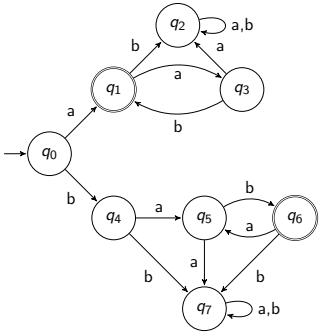
- $\pi = \{C_0, \ldots, C_{k-1}\}$ will denote the current partition.
- An *index* $i \in \{0, \ldots, k-1\}$ is called a *colour*, and the partition *cell* C_i is called the corresponding *colour class*.
- $q_i \equiv_\pi q_j$ denotes that the states q_i and q_j are in the same cell of π (so they currently have the same colour).

Throughout, our partition π satisfies the following *invariant:*

Invariant

If $q_i \not\equiv_\pi q_j$, then there exists a word w that distinguishes q_i from q_j.

So, we always have a partition that is *refined* by the *partition into equivalent states* that we are looking for.

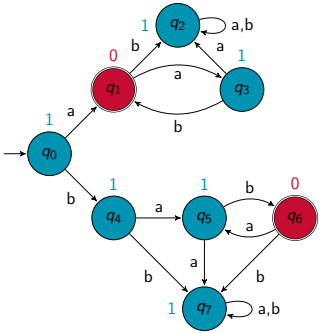


We will start with the partition $\{F, Q \backslash F\}$, since:

Observation

The partition $\{F, Q \backslash F\}$ satisfies the invariant.

Proof: The empty word λ distinguishes accepting states from non-accepting states. $\square$



We will start with the partition $\{F, Q \backslash F\}$, since:

Observation

The partition $\{F, Q \backslash F\}$ satisfies the invariant.

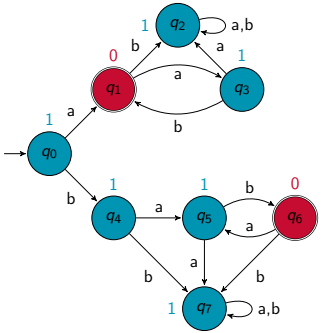
Proof: The empty word λ distinguishes accepting states from non-accepting states. $\square$

Refining Operations

Lemma (Operation $\textsc{Refine}(C, x)$)

Let C be a colour class in the current partition, let $x \in \Sigma$ be a symbol, and let $A \subseteq Q$ be all states q_i with $\delta(q_i, x) \in C$. Then replacing each cell D in the current partition by $D_A = D \cap A$ and $D_{\overline{A}} = D \backslash A$, provided both are nonempty, maintains the invariant.

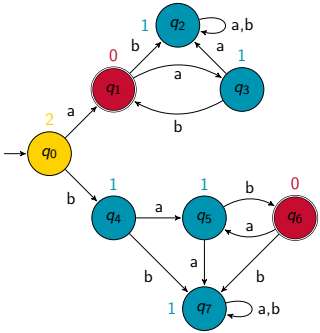
Proof: Consider $q_i \in D_A$ and $q_j \in D_{\overline{A}}$.
Let $q'_i = \delta(q_i, x)$ and $q'_j = \delta(q_j, x)$. So $q'_i \in C$, but $q'_j \notin C$.
So according to the invariant (or: induction assumption), there exists a word w such that *exactly one* of $\hat{\delta}(q'_i, w)$ and $\hat{\delta}(q'_j, w)$ is accepting.
Then also exactly one of $\hat{\delta}(q_i, xw)$ and $\hat{\delta}(q_j, xw)$ is accepting. I.e., the word xw distinguishes q_i and q_j.
Doing this for all state pairs shows that the invariant is maintained. $\square$



Colour:	states:
0	q_1, q_6
1	$q_0, q_2, q_3, q_4, q_5, q_7$

Operation:

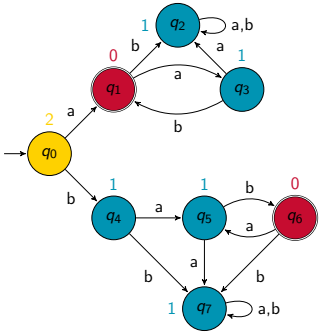
$\textsc{Refine}(C_0, a)$: $A = \{q_0\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_3, q_4, q_5, q_7
2	q_0

Operation:

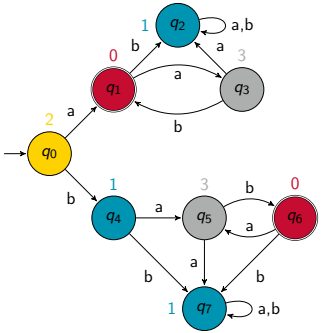
$\textsc{Refine}(C_0, a)$: $A = \{q_0\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_3, q_4, q_5, q_7
2	q_0

Operation:

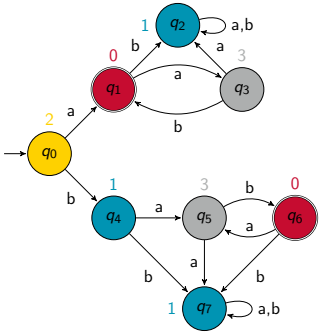
$\textsc{Refine}(C_0, b)$: $A = \{q_3, q_5\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

Operation:

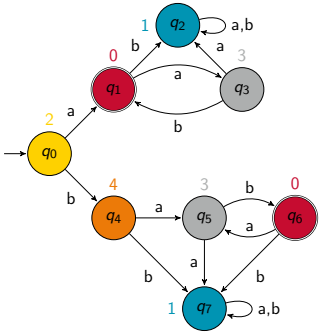
$\textsc{Refine}(C_0, b)$: $A = \{q_3, q_5\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

Operation:

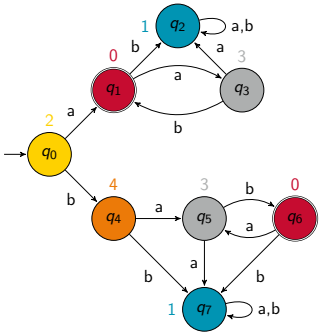
$\textsc{Refine}(C_3, a)$: $A = \{q_1, q_4, q_6\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

Operation:

$\textsc{Refine}(C_3, a)$: $A = \{q_1, q_4, q_6\}$, so colour 1 splits up.



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

Operation:

We cannot refine further: the partition is now *stable*.

- The operation denoted by $\textsc{Refine}(C, x)$ is called the *refining operation for C and x*.
- By $\textsc{Refine}(C)$ we denote the operation that applies $\textsc{Refine}(C, x)$ for all $x \in \Sigma$.
- Note that the operation $\textsc{Refine}(C, x)$ *has no effect* if for all colour classes D, either $D \cap A = \emptyset$ or $D \setminus A = \emptyset$. (with A as defined before.)

Definition

Let C be a colour class of the current partition π. We say that π is stable for C *if the operation $\textsc{Refine}(C)$ has no effect.*
$\pi = \{C_0, \ldots, C_{k-1}\}$ *is* stable *if π is stable for all C_i.*

Basic algorithm idea:

- Start with the partition $\pi = \{F, Q \backslash F\}$.
- As long as it is possible, apply refining operations $\textsc{Refine}(C, x)$ to the partition, for a current colour class C and symbol x.
- Terminate when a stable partition is obtained.

Theorem

The algorithm terminates with the (unique) partition into equivalent states.

Proof: In the handout version of the slides and in the reader (on Canvas). $\square$

Theorem

The algorithm terminates with the (unique) partition into equivalent states.

Proof: Let π^A be the partition given by the algorithm, and π^L be the partition into equivalent states. We show that $\pi^A = \pi^L$, by proving two directions:

π^L *refines* π^A*:*
The initial partition $\{F, Q \backslash F\}$ satisfies the invariant (Observation),and refining operations maintain the invariant (Lemma).

So the final partition π^A satisfies the invariant, and is therefore refined by π^L.

(Proof, other direction:)

π^A *refines* π^L*:*

Suppose to the contrary that there exists a word w that distinguishes two states q_i and q_j with $q_i \equiv_{\pi^A} q_j$.

Choose w, q_i and q_j such that w is a *shortest word* with these properties.

Clearly, $|w| \geq 1$ since we start with the partition $\{F, Q \backslash F\}$.

So let $w = xv$ for $x \in \Sigma$, and let $q_i' = \delta(q_i, x)$ and $q_j' = \delta(q_j, x)$.

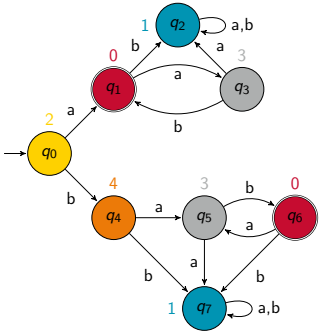
Since π^A is stable, there exists a colour class C' such that $q_i', q_j' \in C'$.

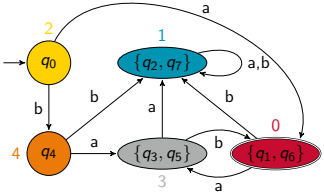
However, the word v distinguishes q_i' from q_j', and v is shorter than w, which contradicts the choice of w.

(This is an induction proof disguised as a proof-by-contradiction. A very useful proof technique!) $\square$

Minimization of DFAs

- The algorithm terminates with partition cells equal to equivalent states.
- *Make every equivalence class into one state of a new DFA.*
- Transitions are easily deduced. (If it is not obvious how to add these, then the partition is not stable yet!)





Summary of DFA Minimization / Equivalence Testing

- This algorithm can only be used for DFAs, not for NFAs!
- Using this algorithm, we can:
  - minimize a DFA, and
  - test whether two DFAs are equivalent.
    (Either: apply this method to both DFAs simultaneously and check whether the start states are equivalent, or: check whether the resulting minimum DFAs are isomorphic, taking edge labels into account.)
- Minimizing a DFA this way yields the *unique* minimum DFA for the language.
  (Details: Myhill-Nerode Theorem.)
- There may however be a smaller NFA for the language!
- This algorithm is different from the one in the L&M lecture/book.
  - The algorithm from the L&M lecture has complexity $O(mn)$.
  - We presented *Hopcroft's algorithm* that has a complexity $O(m \log n)$.

One question remains:

Q: How can we best implement this algorithm? ?

A: Do we need not to check every colour class C_i after every refining operation!
Firstly:

Observation

If π' refines π, and π is stable for C, then π' is stable for C.

But we can do better!

Key idea: If the current partition is stable for a colour class C, and a refining operation splits C into C_A and C_B, then we only have to check one of C_A and C_B!

More precisely

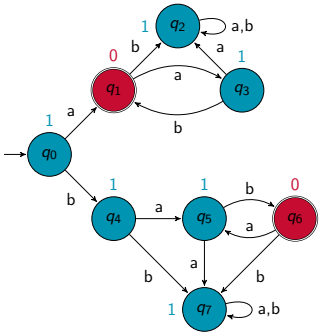
Lemma

Let $C \subseteq Q$ be partitioned into $C_0, \ldots, C_{\ell-1}$.
For any $a \in \{0, \ldots, \ell-1\}$: if a partition π is stable for C and for C_b for every $b \in \{0, \ldots, \ell-1\} \setminus \{a\}$, then π is stable for C_a as well.

Proof: Suppose to the contrary that π is not stable for C_a, so there exist q_i and q_j with $q_i \equiv_\pi q_j$ and a symbol x such that w.l.o.g. $\delta(q_i, x) \in C_a$ but $\delta(q_j, x) \notin C_a$.
Let $q_i' = \delta(q_i, x)$ and $q_j' = \delta(q_j, x)$.
Since π is stable for C and since $C_a \subseteq C$, it holds that $q_i', q_j' \in C$.
So there exists a b with $b \neq a$ such that $q_i' \notin C_b$ and $q_j' \in C_b$ (since $C_0, \ldots, C_{\ell-1}$ is a partition of C).
Then π is also not stable for C_b, a contradiction. $\square$

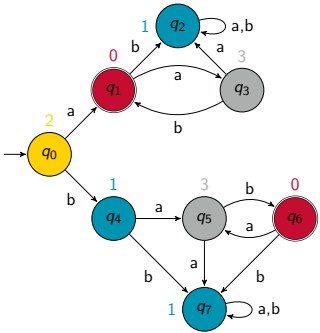
Algorithm 1: Fast Partition/Colour Refinement

- Start with $\pi = \{F, Q \backslash F\} = \{C_0, C_1\}$.
- Maintain a *queue* (or stack) of *colours* that will be used for refining operations, starting with $\textsc{Queue} = [0]$.
- Whenever a refining operation splits a colour class C_i into two new classes: One of them keeps colour i, and the other receives the smallest unused colour ℓ.
  - If $i \in \textsc{Queue}$: add ℓ to the queue.
  - If $i \notin \textsc{Queue}$: add either i or ℓ to the queue, depending on which of the new colour classes is the *smallest*.
- If the queue is empty, the algorithm terminates (with a stable coloring!).



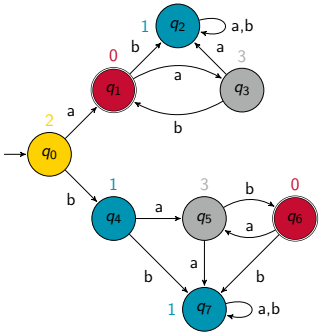
Colour:	states:
0	q_1, q_6
1	$q_0, q_2, q_3, q_4, q_5, q_7$

QUEUE: 0



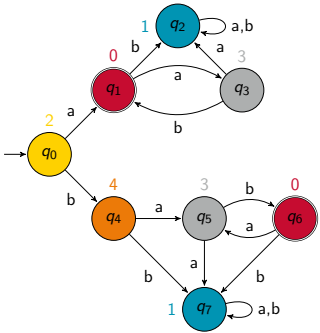
Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

QUEUE: $\emptyset$,2,3



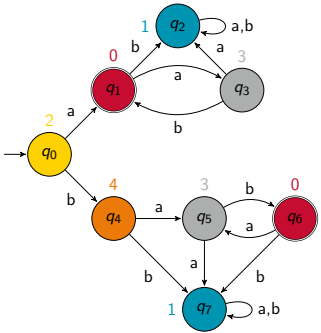
Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

QUEUE: $\emptyset$,$\cancel{2}$,3



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

QUEUE: $\cancel{0}$,$\cancel{2}$,$\cancel{3}$,4



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

QUEUE: $\cancel{0}, \cancel{2}, \cancel{3}, \cancel{4}$

Theorem

Algorithm 1 terminates with a partition into equivalent states.

Proof: Similar to the previous theorem proof (in handout version on Canvas), but now also using the previous lemma. $\square$

The key to the improved complexity is the following lemma:

Lemma

In Algorithm 1, every state is used at most $\log_2 n$ *times in a refining operation, where* $n = |Q|$.

Lemma

In Algorithm 1, every state is used at most $\log_2 n$ *times in a refining operation, where* $n = |Q|$.

Proof: Consider a state q.
Let $C_0, C_1, \ldots, C_\ell$ be all different colour classes that contain q, that have been used in refining operations (they were element of the queue at one time).
So $\{q\} \subseteq C_\ell \subseteq C_{\ell-1} \subseteq \ldots \subseteq C_0$.
Then because of our queueing rule:
For every i, C_{i+1} is obtained from splitting up C_i into at least two parts, and C_{i+1} is not the largest part, so $2|C_{i+1}| \leq |C_i|$.
It follows that $|C_0| \geq 2^\ell$.
Obviously $|C_0| \leq n$, so $\ell \leq \log_2 n$. $\square$

Definition

For a state set C, denote by $E^-(C)$ all transitions into C.

Next, we will prove the following lemma:

Lemma

Algorithm 1 can be implemented such that the operation $\textsc{Refine}(C)$ *takes time* $O(|E^-(C)|)$.

Combining the previous two lemma's, the main theorem follows:

Theorem

For any DFA $D = (Q, \Sigma, \delta, q_0, F)$, the partition into equivalent states can be computed in time $O(m \log n)$, where $n = |Q|$, and m $(\leq n|\Sigma|)$ is the total number of transitions.

Theorem

For any DFA $D = (Q, \Sigma, \delta, q_0, F)$, the partition into equivalent states can be computed in time $O(m \log n)$, where $n = |Q|$, and m $(\leq n|\Sigma|)$ is the total number of transitions.

Proof: The complexity of Algorithm 1 is dominated by the complexity of the refining operations. (We may ignore the various initialization steps.)

We use the previous two lemma's to bound that complexity.

First, sum over all refining operations $\textsc{Refine}(C)$ that have been used:

$$\sum_C O(|E^-(C)|) = \sum_{q \in Q} O(|E^-(\{q\})|) \cdot (\#\text{times } q \text{ has been used in ref. op}) \leq$$

$$\sum_{q \in Q} O(|E^-(\{q\})|) \cdot \log_2 n = O(|E^-(Q)|) \cdot \log_2 n = O(m \log n).$$

$\square$

Only one thing remains to be done:
Proving that the operation $\textsc{Refine}(C)$ can be done in time $O(|E^-(C)|)$, using the proper implementation.

Observe: It suffices to show that the operation $\textsc{Refine}(C, x)$ can be done in time $O(|E^-(C, x)|)$, where $E^-(C, x)$ denotes the set of x-transitions into C.

One key ingredient for the proof: *Doubly Linked Lists (DLLs)*.

Proposition

For a DLL of length p:

- *all elements can be listed in time $O(p)$ (in order),*
- *a single DLL item can be deleted in time $O(1)$, and*
- *an element can be added to the start or end of the list in time $O(1)$.*

Note for implementation: We don't need the ordered property of a DLL. A Python *set* has similar average case complexity.

Data structures

- For each state q and symbol x, we have a list $N_x^-(q)$ of all states q' with $\delta(q', x) = q$. (All incoming x-transitions)

At any time during the computation, we maintain:

- A *queue* $\textsc{Queue}$ that contains the colours (numbers) that need to be considered for refining operations. (No duplicates!)
- For each colour $i \in \{0, \ldots, k-1\}$:
  - a *DLL* C_i containing all states with current colour i.
  - a *boolean variable* $\textsc{InQueue}[i]$ that indicates whether colour i is in $\textsc{Queue}$.
- For each state $q \in Q$:
  - A pointer to the DLL item that represents q.
  - The current colour of q: $\textsc{colour}[q]$.

Implementing Fast Refining Operations

Operation $\textsc{Refine}(C, x)$ in time $O(t)$, with $t = |E^-(C, x)|$:

- In time $O(t)$, loop over all $q \in C$ and $q' \in N_x^-(q)$, and compute:
  - A list L of all colours i such that C_i contains a q' with $\delta(q', x) \in C$,
  - The number $A[i]$ of such states in C_i.
- In time $O(|L|) \leq O(t)$, loop over L and compute:
  - for each $i \in L$: whether C_i should split up (whether $A[i] < |C_i|$).
  - if so, choose a new colour ℓ, and update $\textsc{Queue}$ by adding i or ℓ.
- In time $O(t)$, loop over all $q \in C$ and $q' \in N_x^-(q)$, and:
  - Update the colours of the states (update the DLLs, and update $\textsc{colour}[q']$).
    (Key fact: moving a state from one DLL to another can be done in time $O(1)$.)

This assumes that $|C| = O(t)$. Depending on the used data structures, the red loops need to treat $\textsc{Refine}(C, x)$ for all x simultaneously to obtain $O(|E^-(C)|)$ running time.

Beyond DFA Minimization: The GI Problem

The same techniques can be used to speed up the colour refinement algorithm for the GI problem, to $O(m \log n)$, where $n = |V(G)|$ and $m = |E(G)|$.

Outline/hints:

Invariant

For a graph G, we have a partition π of G that is refined by the coarsest stable partition of G.

Lemma (Refining operation for GI: $\textsc{Refine}(C, i)$)

Let C be a colour class in the current partition, $i \geq 0$ be an integer and let A denote the set of vertices with exactly i neighbours in C. Replacing each cell D in the current partition by $D_A = D \cap A$ and $D_{\overline{A}} = D \setminus A$, provided both are non-empty, maintains the invariant.

(Just implementing this refinement operation does not suffice & slower in most cases!)

Beyond DFA Minimization: The GI Problem

Basic algorithm:

- Start with a colouring (e.g., uniform at the root of the branching tree).
- As long as it is possible, apply refining operations $\textsc{Refine}(C)$ to the coloring, for a current colour class C. ($\textsc{Refine}(C)$ applies $\textsc{Refine}(C, i)$ for all $i > 0$.)
- Terminate when a stable colouring is obtained.

Be aware of:

- The data structures that you use and what the complexity is of the methods (of those data structures) that you need.
- How you loop over the vertices of the graph during the refinement.
- $\textsc{Refine}(C)$ should run in $O(t)$, with t the number of edges incident with a vertex in C.

You can also review your existing code for these things, if you do not implement FCR.

Computation Times in Practice

This improvement is particularly useful for graphs containing long paths as subgraphs. Below are some path-based benchmark instances, which can also be found on canvas.
Computation times:

Instance:	Basic colour ref.:	Fast colour ref.:
threepaths160	0.07s	0.03s
threepaths320	0.26s	0.03s
threepaths640	0.78s	0.05s
threepaths1280	3.59s	0.07s
threepaths2560	15.40s	0.11s
threepaths5120	105.98s	0.21s
threepaths10240	537.74s	0.40s

(Includes time used to read the graphs.)

Even if your implementation is 100× slower this is still a great improvement for the large graphs!

Keep track of the running time improvements between versions of your algorithm. **Use the threepaths(_complement) instances for this fast refinement algorithm!!!**

Computation Times in Practice

This improvement is particularly useful for graphs containing long paths as subgraphs. Below are some path-based benchmark instances, which can also be found on canvas.
Computation times:

Instance:	Basic colour ref.:	Fast colour ref.:
threepaths_complement40	0.06s	0.03s
threepaths_complement80	0.45s	0.10s
threepaths_complement160	3.56s	0.35s
threepaths_complement320	22.22s	1.37s
threepaths_complement640	230.75s	4.17s
threepaths_complement1280	–	17.53s
threepaths_complement2560	–	165.94s

(Includes time used to read the graphs.)

Even if your implementation is 100× slower this is still a great improvement for the large graphs!

Keep track of the running time improvements between versions of your algorithm. **Use the threepaths(_complement) instances for this fast refinement algorithm!!!**

Project: This Week

- Thursday, project session for questions about your implementation and help with ideas, bugs, etc.
- You can use the ideas from this lecture in the coming week(s) to implement fast colour refinement (also check the reader!).
- As the previous table shows, it is easy to distinguish programs that correctly implement fast colour refinement from those that don't!